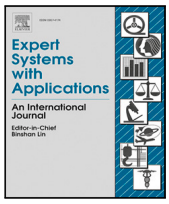
Expert Systems With Applications 284 (2025) 127821



Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa





Joint computation offloading and service caching in Vehicular Edge Computing via a dynamic coevolutionary multiobjective optimization algorithm

Qijie Qiu [a], Yulong Ye [b], Lingjie Li [c], Zhijiao Xiao [a,*], Qiuzhen Lin [a], Zhong Ming [a,c,d,*]

[a] College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, China
[b] School of Computer Science, University of Birmingham, Birmingham, UK
[c] College of Big Data and Internet, Shenzhen Technology University, Shenzhen, China
[d] Guangdong Laboratory of Artificial Intelligence and Digital Economy (SZ), Shenzhen, China

ARTICLE INFO

Keywords:
Vehicular Edge Computing
Computational offloading
Service caching
Dynamic multiobjective optimization
Coevolutionary optimization

ABSTRACT

With the development of autonomous driving technologies, Vehicular Edge Computing (VEC) has become a powerful computing paradigm that improves vehicle performance and edge resource efficiency by introducing Computation Offloading and Service Caching (CO-SC). However, due to the mobility and dynamics of vehicular networks, the problem of jointly optimizing CO-SC under dynamic VEC systems is extremely challenging, as CO-SC decisions must be adjusted in real-time based on the current location and state of the vehicular network. To alleviate this problem, in this paper, we first formulate the joint CO-SC problem as a dynamic multi-objective optimization problem with the objective of simultaneously minimizing delay, energy consumption, and load balance. Then, this paper proposes a dynamic coevolutionary multi-objective optimization algorithm (DCMOA) to search for optimal CO-SC decisions under dynamic VEC systems. Specifically, we design a multi-population coevolutionary approach following the idea of *divide-and-conquer* to simplify the complexity of target problem, and then utilize knowledge transfer between subpopulations to coevolve each subpopulation. Besides, we devise an effective dynamic feedback mechanism to achieve timely feedback on environmental changes. Extensive simulations on real-world datasets of telecom base stations and cab trajectories in *Shanghai, China* show that our proposed method outperforms several state-of-the-art peer methods.

1. Introduction

With the development of artificial intelligence, autonomous driving technology is attracting increasing attention as an innovative area in modern transportation systems (Zhao et al., 2024). However, the services and applications of autonomous driving impose stringent performance requirements, particularly in terms of delay/latency, as they integrate sensor data, map information, and real-time environmental analysis to realize unmanned driving operations (Tian et al., 2021). To meet the unprecedented surge in data traffic and increasingly stringent vehicle application requirements, vehicle edge computing (VEC) has emerged as an emerging computing model that offloads computational tasks to edge servers (ESs) for reducing the computational burden of vehicles (Farimani et al., 2024).

In the VEC systems, due to the constraints of network conditions and task latency, it remains a great challenge to make an optimal offloading decision for each computational task, which is known as the computation offloading problem (Wang et al., 2022). On the other hand, services and data required to perform computational tasks need to be downloaded from the cloud server and cached to the ES. However, due to the limited storage resources of ES, it may be infeasible for ES to cache all types of services simultaneously. Therefore, how to selectively cache services while considering spatial and temporal resources is also a great challenge, which is known as the service caching problem (Li, Zhang et al., 2021). To maximize system performance and resource utilization efficiency, the joint computation offloading and service caching (CO-SC) optimization problem for VEC systems has been widely discussed in academia and industry (Zhou et al., 2023). Fig. 1 illustrates CO-SC in VEC systems. Apparently, the CO-SC problem has proven to be an NP-hard problem with multiple optimization objectives involving delay and energy consumption (Zhou et al., 2022). However, load balance has rarely been addressed in the existing studies. In practice, an imbalanced load distribution can shorten the lifespan of overloaded

* Corresponding authors.
E-mail addresses: qiuqijie2021@email.szu.edu.cn (Q. Qiu), yxy382@student.bham.ac.uk (Y. Ye), lilingjie@sztu.edu.cn (L. Li), cindyxzj@szu.edu.cn (Z. Xiao), qiuzhlin@szu.edu.cn (Q. Lin), mingz@szu.edu.cn (Z. Ming).

https://doi.org/10.1016/j.eswa.2025.127821
Received 27 March 2024; Received in revised form 30 October 2024; Accepted 18 April 2025
Available online 2 May 2025
0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

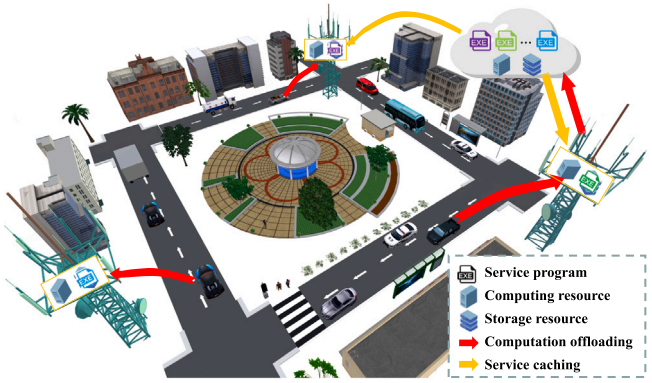


Fig. 1. Illustration of CO-SC in a VEC system: the vehicle can offload computational tasks to ES, which typically caches some service programs for solving the corresponding tasks.

edge servers, leading to economic losses and service degradation. To address this, load balance is incorporated alongside delay and energy consumption as key optimization objectives in this paper.

In this regard, several existing studies (Li et al., 2022; Pham et al., 2020) have shown that evolutionary algorithms are effective methods for solving CO-SC problems due to their global search capability, ease of implementation, and flexibility (Song et al., 2023; Yang et al., 2023). Specifically, these methods first transform the CO-SC problem into a multi-objective optimization problem (MOP), and then use various evolutionary algorithms to explore the optimal CO-SC decision scheme in the VEC system. However, the vast majority of previous studies (Hosny et al., 2023) have only addressed the CO-SC problem under a static environment, neglecting the influence of vehicle mobility and real-time environmental dynamics on optimal decision-making. The approach of these articles are inefficient and irrational in real-world scenarios, since vehicle mobility and real-time environmental changes are the most essential characteristics for a VEC system. For example, when a vehicle moves from one region to another, its computation offloading decision should also be changed from the ES of the previous region to that of the current region to minimize the transmission delay. Accordingly, the service caching decisions of ESs should also be refreshed to improve the quality of service. Therefore, it is more significant and challenging to solve the CO-SC problem and explore its optimal decision scheme in dynamic VEC systems (Zhang et al., 2021).

Based on the above analysis, this paper considers the CO-SC problem in dynamic VEC systems and then formulates it as a dynamic multi-objective optimization problem (DMOP) with the purpose of obtaining the optimal CO-SC decision that minimizes delay, energy consumption, and load balance simultaneously in dynamic environments. Nevertheless, there are two major challenges in designing an effective optimization method to solve the above constructed problem:

1. Frequent environmental changes require that the optimization algorithm must quickly find the appropriate solution before the environmental changes, however, it is very difficult to identify the optimal solution in a short period of time.
2. Dramatic environmental changes will inevitably lead to the fact that previously found high-quality solutions may no longer be optimal in the new environment, resulting in a significant cost for the optimization algorithm to re-explore the optimal solution in the new environment.

To address the above challenges, this paper proposes a dynamic coevolutionary multi-objective optimization algorithm, called DCMOA. First, DCMOA designs a multi-subpopulation coevolutionary approach to quickly find high-quality CO-SC solutions prior to environmental change. Second, DCMOA develops a dynamic feedback mechanism that employs three different feedback strategies to efficiently response to the environmental changes. To the best of our knowledge, we are the first work to construct the CO-SC problem in the dynamic VEC system as a DMOP and design an efficient dynamic coevolutionary algorithm to solve the problem. In summary, the main contributions of this paper are summarized below.

- We are the first work to formulate the joint computation offloading and service caching problem (CO-SC) in dynamic VEC systems as a DMOP with the goal of simultaneously minimizing delay, energy consumption, and load balance in dynamic environments.
- To improve the optimization efficiency, we design a multi-subpopulation coevolutionary approach, where each subpopulation focuses on optimizing a specific combination of objectives, thus simplifying the complexity of the problem and accelerating the convergence speed via knowledge transfer between subpopulations.
- To cope with dramatic environmental changes, we devise an efficient dynamic feedback mechanism that integrates diversity-based, memory-based, and prediction-based dynamic feedback methods, aiming to rapidly acquire high-quality subpopulations in new environments.
- To evaluate the performance of the DCMOA, we use real-world datasets of telecom base stations and cab trajectories in Shanghai, China in simulation experiments. Extensive simulation results show that DCMOA outperforms several baseline methods.

The rest of the paper is organized as follows. Section 2 introduces the background and the related work of this work. Section 3 introduces the problem formulation. Section 4 presents the details of the proposed method. Section 5 provides simulation results and analysis. Section 6 discusses some limitations of this work. Section 7 summarizes the key findings of this paper and discusses some future research directions.

2. Background and related work

2.1. Dynamic multi-objective optimization

According to different dynamic constraint problems in the real world, DMOPs can be formulated in various ways (Gao & Xu, 2024; Hu et al., 2024; Wang et al., 2024). In general, the mathematical form of a DMOP can be expressed as follows:

$$\begin{aligned} &\min F(\mathbf{x},t)=[f_1(\mathbf{x},t),f_2(\mathbf{x},t),\ldots,f_m(\mathbf{x},t)]^T,\\ &\text{s.t. } \mathbf{x}\in\Omega, \end{aligned} \tag{1}$$

where $\mathbf{x}=(x_1,x_2,\ldots,x_n)$ is an n-dimensional decision vector bounded in the decision space Ω, t is the time parameter, and $(f_1,f_2,\ldots,f_m)$ are m objective functions.

The key to solving the DMOP is to find the dynamic Pareto optimal set or the dynamic Pareto optimal front based on dynamic Pareto dominance, as defined below.

Definition 1 (*Dynamic Pareto Dominance*)**.** At time t, a decision vector $\mathbf{x}_1$ is said to Pareto dominate another decision vector $\mathbf{x}_2$, denoted by $\mathbf{x}_1 \succ_t \mathbf{x}_2$, if and only if

$$\begin{cases} \forall i=1,\ldots,m, & f_i(\mathbf{x}_1,t)\le f_i(\mathbf{x}_2,t),\\ \exists j=1,\ldots,m, & f_j(\mathbf{x}_1,t)< f_j(\mathbf{x}_2,t). \end{cases} \tag{2}$$

Definition 2 (*Dynamic Pareto Optimal Set, **DPOS***)**.** A dynamic Pareto optimal set at time t, denoted as **DPOS**(t), includes all solutions that are not dominated by any other solutions at time t, defined as follows:

$$\mathbf{DPOS}(t)=\{\mathbf{x}^* \mid \nexists \mathbf{x},\ \mathbf{x}\succ_t \mathbf{x}^*\}. \tag{3}$$

Definition 3 (*Dynamic Pareto Optimal Front, **DPOF***). A dynamic Pareto optimal front at time t, denoted as **DPOF**(t), is the mapping of the solutions in **DPOS**(t) in the objective space, expressed as below:

$$\mathbf{DPOF}(t) = \{ F(\mathbf{x}, t) \mid \mathbf{x} \in \mathbf{DPOS}(t) \}. \tag{4}$$

2.2. Related work

This section reviews related work on computation offloading, service caching, and joint CO-SC optimization, and clarifies the motivation of our work.

2.2.1. Computation offloading

In recent years, computation offloading in edge computing has attracted great attention, as it can effectively reduce the processing delay of computation-intensive tasks. For example, Shen et al. (2023) addressed computation offloading in a distributed VEC network, where a task vehicle offloads tasks to energy-harvesting surrounding vehicles. Mirza et al. (2023) addressed the computation offloading problem in a VEC network based on multi-access edge computing and vehicular cloudlets. Inspired by data analysis and observation results in large-scale WiFi systems, Duan et al. (2024) studied the mobile-aware online task offloading problem with adaptive load balancing to minimize the total computational cost. Facing the complex heterogeneous architectures in edge computing, Yuan et al. (2024) proposed an energy-minimized partial computation offloading technique based on an improved hybrid meta-heuristic algorithm that synergistically combines a Metropolis acceptance criterion of simulated annealing with genetic operations. Ameena and Ramasamy (2025) presented a newly developed scheme for task offloading in fog computing based on multiple objectives such as makespan, cost, load, and energy, which was designed by means of the unification of the drawer algorithm and the sine cosine algorithm. In order to achieve efficient distribution and execution of tasks between edge servers and service vehicles, Li et al. (2023) proposed an improved particle swarm genetic algorithm to realize the optimal offloading policy and reduce the task delay.

2.2.2. Service caching

Until now, numerous studies have delved into various issues concerning computation offloading. However, a predominant assumption in most of these studies is that edge nodes are deployed with all required services to handle tasks of any type. Given the limited resources of edge nodes, this assumption becomes untenable. Therefore, service caching emerges as a highly pertinent topic for research. For instance, Dutta et al. (2022) proposed a caching method that balances the distribution of content among network devices and also takes into account the dynamic changes in the popularity of content. In order to improve the performance of application outsourcing in cache-assisted VEC, Tang et al. (2021) proposed a dynamic algorithm based on drift plus penalty term to optimize the response time and energy consumption of the system. In heterogeneous air access networks, Masood et al. (2024) proposed an intelligent and proactive content caching scheme using hierarchical federated learning to maximize caching efficiency while ensuring privacy protection of ground terminals. To cope with the unpredictability of service requests, Liu et al. (2024) proposed a service caching scheme based on deep reinforcement learning enabled by digital twin to obtain the optimal caching strategy for IoT devices. Jiang et al. (2024) proposed a revocable access control scheme with fair incentives for privacy-aware service caching in a VEC network and developed an evolutionary game theoretic model to analyze the effectiveness and fairness of fair incentives in the scheme.

2.2.3. Joint CO-SC optimization

The location of service caching is strongly correlated with computation offloading decisions. Therefore, the joint optimization of computation offloading and service caching is crucial for achieving optimal system performance. For example, exploiting the horizontal cooperation among base stations, Huang et al. (2024) studied the joint service caching and task offloading decision problem and proposed a two-stage optimization framework based on the concept of cooperative community to maximize the benefits of each base station. Fan et al. (2022) proposed a novel resource management scheme with joint computation offloading and service caching to maximize the energy efficiency of all mobile terminals covered by a WiFi cellular heterogeneous network while guaranteeing task processing delay tolerance for each mobile terminal. Xu et al. (2024) investigated the joint task offloading and service caching problem in MEC networks applied to commercial domains to reduce the latency and computational cost of all commercial services and proposed a data-driven hybrid soft actor-critic scheme, where the deep reinforcement learning determines the near-optimal service caching decision and the convex optimization technique computes the optimal offloading decision.

In order to solve the complex CO-SC problem efficiently, researchers have gradually turned to evolutionary algorithms as a promising approach due to its advantages in global search and multi-objective optimization (Cui et al., 2024; Li et al., 2024). In order to obtain a suitable joint optimization strategy for task offloading and service caching, Wang et al. (2023) utilized an improved multi-objective evolutionary algorithm based on decomposition (MOEA/D), which decomposes the multi-objective problem of minimizing latency and energy consumption into multiple single-objective problems by means of the Tchebycheff Weights Aggregation Method, and obtains a set of Pareto-optimal solutions. Gao et al. (2022) formulated the computation offloading and service caching problem as a constrained discrete optimization problem and proposed a hybrid heuristic based on particle swarm optimization to solve the problem in polynomial time. Xiao et al. (2023) proposed an enhanced binary particle swarm optimization algorithm and an improved multi-objective bat algorithm to solve the service caching and computation offloading problems, respectively, thereby minimizing task latency and energy consumption.

2.2.4. Motivation

A comparison between our work and the current work is summarized in Table 1, and the following points of analysis can be observed.

1. As summarized in the first three columns of Table 1, the existing works in the first two categories focus only on computation offloading and service caching, respectively, while overlooking their interdependence. The third category of existing work only addresses the joint CO-SC optimization problem but only in a static environment, neglecting the dynamic nature of vehicle mobility and network conditions. This limitation may lead to performance instability and decreased efficiency of the current algorithms in practical applications. Therefore, it is necessary to design a dynamic optimization algorithm to solve the joint CO-SC problem in a dynamic VEC system.
2. As summarized in the last three columns of Table 1, most existing works usually prioritize task delay or energy consumption as optimization objectives of VEC systems, overlooking the crucial metric of load balance among edge computing nodes. In fact, these approaches achieve lower latency and reduced energy consumption by offloading tasks to the nearest or most resource-rich nodes. However, this strategy may cause load imbalance, shortening the lifetime of overloaded nodes. Therefore, it is essential to develop a multi-objective optimization algorithm that simultaneously optimizes delay, energy consumption, and load balancing within the VEC system.

Table 1
Comparison between our work and the current work.

Reference	Computation offloading	Service caching	Dynamic environments	Task delay	Energy consumption	Load balance
Shen et al. (2023)	√		√	√	√	
Mirza et al. (2023)	√		√	√		
Duan et al. (2024)	√			√	√	
Yuan et al. (2024)	√			√	√	
Ameena and Ramasamy (2025)	√		√	√	√	
Li et al. (2023)	√			√	√	
Dutta et al. (2022)		√	√		√	
Tang et al. (2021)		√			√	
Masood et al. (2024)		√	√		√	
Liu et al. (2024)		√	√		√	
Jiang et al. (2024)		√	√			
Huang et al. (2024)	√	√		√		
Fan et al. (2022)	√	√		√	√	
Xu et al. (2024)	√	√		√	√	
Li et al. (2024)	√	√		√	√	
Cui et al. (2024)	√	√		√	√	
Wang et al. (2023)	√	√		√	√	
Gao et al. (2022)	√	√		√		
Xiao et al. (2023)	√	√		√	√	
Our work	√	√	√	√	√	√

Table 2
Main notations.

Symbol	Definition
V, I	A set of vehicles and the number of vehicles
ES, J	A set of ESs and the number of ESs
S, K	A set of services and the number of services
t, T	A time slot and the number of time slots
d_i, c_i,	The task data size and CPU cycles of v_i
O_j	The storage resources of es_j
x_{ij}	Whether the task of v_i is offloaded to es_j
l_{ik}	Whether the task of v_i belongs to s_k
y_{jk}	Whether s_k is cached on es_j
a_i^{edge}	Whether v_i chooses edge computing
a_i^{local}	Whether v_i chooses local computing
a_i^{cloud}	Whether v_i chooses cloud computing
f_{ij}	The computational resources of es_j allocated to v_i
F_i^{local}	The computational resources of v_i
F_j^{edge}	The computational resources of es_j
F^{cloud}	The computational resources of cloud
R_{ij}	The wireless transmission rate between v_i and es_j
B_{ij}	The bandwidth between v_i and es_j
B^{cloud}	The bandwidth between ES and cloud
U_k	The service data size of s_k

Based on the above analysis, this paper considers the CO-SC problem in dynamic VEC systems and formulates it as a DMOP aiming to minimize delay, energy consumption, and load balance simultaneously. To efficiently solve the above DMOP, a novel dynamic co-evolutionary multi-objective optimization algorithm, namely DCMOA, is proposed in this paper, which consists of a multi-subpopulation co-evolutionary approach for discovering high-quality solutions and a dynamic feedback mechanism for responding to changes in the environment.

3. Problem formulation

3.1. VEC system design

As illustrated in Fig. 1, we consider a general vehicular network consisting of an edge network, the cloud, and a number of moving vehicles. Assume that $ES = \{es_1, es_2, \ldots, es_J\}$ is a set of edge servers, and $V = \{v_1, v_2, \ldots, v_I\}$ is a set of vehicles. Each vehicle has only one indivisible computational task, and each computational task belongs to only one service in the VEC system. Specifically, a binary matrix $\mathbf{X}$ of size $I \times J$ is represented as a mapping relationship between vehicles and ES for computation offloading, where each element $x_{ij} = 1$ indicates that the computational task of v_i is offloaded on es_j, otherwise $x_{ij} = 0$. Assume that $S = \{s_1, s_2, \ldots, s_K\}$ is a set of application services. If the computational task of v_i belongs to s_k, it is denoted as $l_{ik} = 1$, otherwise $l_{ik} = 0$. Moreover, a binary matrix $\mathbf{Y}$ of size $K \times J$ is represented as a mapping relationship between services and ES for service caching, where $y_{jk} = 1$ means that s_k is cached on es_j, otherwise $y_{jk} = 0$. Due to the mobility of vehicles, the CO-SC policies should be updated over time in dynamic VEC systems. Therefore, time is divided into a set of discrete time slots, indexed by $\{1, 2, \ldots, t, \ldots, T\}$. For ease of description, the main symbols in the system are summarized in Table 2.

3.2. Delay model

3.2.1. Edge computing

Assume that the vehicle communicates with the ES through wireless links in time slot t. In this case, the signal-to-interference-plus-noise ratio (SINR) (Yang et al., 2020) between v_i and es_j in time slot t can be calculated by:

$$SINR_{ij}(t) = \frac{P_{ij}(t)G_{ij}(t)}{\varepsilon_{ij}(t)\zeta_{ij}(t)\sigma^2(t)}, \tag{5}$$

where $\sigma^2(t)$ refers to the power of the white Gaussian noise, $\varepsilon_{ij}(t)$ is the path loss, $G_{ij}(t)$ denotes the average channel gain, $P_{ij}(t)$ represents the average channel transmission power of es_j, and $\zeta_{ij}(t)$ is the positional distance between v_i and es_j in time slot t. The wireless transmission rate in time slot t is calculated based on Shannon's formula, as follows:

$$R_{ij}(t) = B_{ij} \times \log_2\left(1 + SINR_{ij}(t)\right), \tag{6}$$

where B_{ij} is the bandwidth between v_i and es_j.

When the vehicle decides to offload the task to the ES, there will exist delays in the interaction between the vehicle and ES, including transmission delay and computational delay. First, the transmission delay $TD_{ij}(t)$ is influenced by the data size of the task $d_i(t)$ and the transmission rate $R_{ij}(t)$ between the vehicle and the ES, and is calculated as follows:

$$TD_{ij}(t) = \frac{d_i(t)}{R_{ij}(t)} + \left(1 - y_{jk}(t)\right) \times \frac{U_k}{B^{Cloud}}, \tag{7}$$

where the second term indicates the delay in downloading the service corresponding to the task to the ES if the service is not cached on the ES, i.e., $y_{jk}(t) = 0$. Note that U_k indicates the data size of the service, and B^{Cloud} is the bandwidth between the ES and the cloud. Then, after the transmission is completed, the computational delay of the task $CD_{ij}(t)$

being executed on the ES depends on the CPU cycles $c_i(t)$ required for task execution and is calculated as:

$$CD_{ij}(t) = \frac{c_i(t)}{f_{ij}(t)}, \tag{8}$$

where $f_{ij}(t)$ is the computational resource allocated by es_j to v_i. Note that the fair allocation approach mentioned in Zhou and Zhang (2021) is employed to allocate computational resources on ES. Finally, the total delay of edge computing is the sum of transmission delay and computation delay when offloaded to a particular ES, i.e., $x_{ij}(t) = 1$, and is calculated as:

$$D_i^{edge}(t) = \sum_j^J \Big(x_{ij}(t) \times \left(TD_{ij}(t) + CD_{ij}(t) \right) \Big). \tag{9}$$

3.2.2. Local computing

If the computational resources of the vehicle are sufficient, the task can be executed locally instead of being offloaded to ESs or the cloud, which avoids the impact of wireless communication aspects. For simplification, it is assumed that the vehicle itself caches the services required for the task. Therefore, the delay for local computing in time slot t, which contains only the computational delay, is modeled as follows:

$$D_i^{local}(t) = \frac{c_i(t)}{F_i^{local}}, \tag{10}$$

where F_i^{local} is the computational resources of the vehicle.

3.2.3. Cloud computing

If ES becomes a hotspot or the computational task is heavy, it may result in insufficient resources for edge computing and local computing. In this case, the vehicle has to offload tasks to the cloud via the nearest ES to obtain sufficient computational resources. The delay of cloud computing consists of transmission delay and computational delay, which is calculated as:

$$D_i^{cloud}(t) = \frac{d_i(t)}{\hat{R}_{ij}(t)} + \frac{d_i(t)}{B^{cloud}} + \frac{c_i(t)}{F^{cloud}}, \tag{11}$$

where the first term represents the transmission delay of the task offloading from the vehicle to the nearest ES, the second term is the transmission delay between the ES and the cloud center, and the last term is the computational latency of the task on the cloud center. Since the output data are typically much smaller than the input task size, the time delay of the output return process can be ignored.

In the proposed VEC system, which incorporates both horizontal collaboration among edge servers (ESs) and vertical cloud–edge-device collaboration (Hong, 2021), vehicles can select the most suitable computing mode based on their specific requirements. The binary vector $\mathbf{A}_i = \left\{ a_i^{edge}, a_i^{local}, a_i^{cloud} \right\}$ is denoted as the chosen computational mode for offloading, where $a_i^{edge} = 1$, $a_i^{local} = 1$, and $a_i^{cloud} = 1$ represent that v_i chooses edge computing, local computing, or cloud computing, respectively. Note that each vehicle can only choose one computing mode, i.e., $a_i^{edge} + a_i^{local} + a_i^{cloud} = 1$ and $a_i^{edge}, a_i^{local}, a_i^{cloud} \in \{0, 1\}$. In summary, the total delay of all vehicles at time slot t is the aggregate of the delays associated with the computing modes selected by each vehicle and is expressed as:

$$\begin{aligned} Delay(t) = \sum_i^I \Big(& a_i^{edge}(t) \times D_i^{edge}(t) \\ & + a_i^{local}(t) \times D_i^{local}(t) \\ & + a_i^{cloud}(t) \times D_i^{cloud}(t) \Big). \end{aligned} \tag{12}$$

3.3. Energy consumption model

To simplify the calculations, this work considers only the energy consumption during task execution. Following Zheng et al. (2022), the energy consumption of edge computing ($E_i^{edge}(t)$), local computing ($E_i^{local}(t)$), and cloud computing ($E_i^{cloud}(t)$) are respectively calculated as:

$$E_i^{edge}(t) = \kappa \left(f_{ij}(t) \right)^2 c_i(t), \tag{13}$$

$$E_i^{local}(t) = \kappa \left(F_i^{local} \right)^2 c_i(t), \tag{14}$$

$$E_i^{cloud}(t) = \kappa \left(F^{cloud} \right)^2 c_i(t), \tag{15}$$

where κ denotes the computing energy efficiency parameter. The sum of the energy consumption selected for each vehicle at time slot t is the total energy consumption of the VEC system and is obtained by:

$$\begin{aligned} Energy(t) = \sum_i^I \Big(& a_i^{edge}(t) \times E_i^{edge}(t) \\ & + a_i^{local}(t) \times E_i^{local}(t) \\ & + a_i^{cloud}(t) \times E_i^{cloud}(t) \Big). \end{aligned} \tag{16}$$

3.4. Load balance model

Load balance can evenly distribute the computational tasks and communication traffic of vehicles among different ESs, thereby avoiding overloading of ESs that may lead to performance degradation or service interruptions. In this paper, the computational load of an ES depends on the amount of computation requested by all the tasks it hosts and is formulated as:

$$L_j(t) = \sum_i^I \left(x_{ij}(t) \times c_i(t) \right). \tag{17}$$

Moreover, the load balance of the VEC system is represented by the standard deviation of the computational load across all ESs, which is formulated as:

$$Load(t) = \sqrt{\frac{1}{J} \sum_j^J \left(\bar{L}(t) - L_j(t) \right)^2}, \tag{18}$$

where $\bar{L}(t)$ is the average computational load of all ESs.

3.5. Problem formulation

In this paper, the CO-SC problem is modeled as a constrained DMOP with the purpose of minimizing delay, energy consumption, and load balance. Mathematically, the formulated problem is defined as follows:

$$F_0 = \min_{\mathbf{A},\mathbf{X},\mathbf{Y}} \begin{cases} P1 = \frac{1}{T} \sum_t^T Delay(t), \\ P2 = \frac{1}{T} \sum_t^T Energy(t), \\ P3 = \frac{1}{T} \sum_t^T Load(t), \end{cases} \tag{19}$$

$$s.t. \quad \sum_j^J x_{ij} = 1, \forall i \in I, \tag{20}$$

$$\sum_k^K l_{ik} = 1, \forall i \in I, \tag{21}$$

$$\sum_k^K (y_{jk} \times U_k) \le O_j, \forall j \in J, \tag{22}$$

$$\sum_i^I f_{ij} \le F_j^{edge}, \forall j \in J, \tag{23}$$

$$a_i^{edge} + a_i^{local} + a_i^{cloud} = 1, \forall i \in I, \tag{24}$$

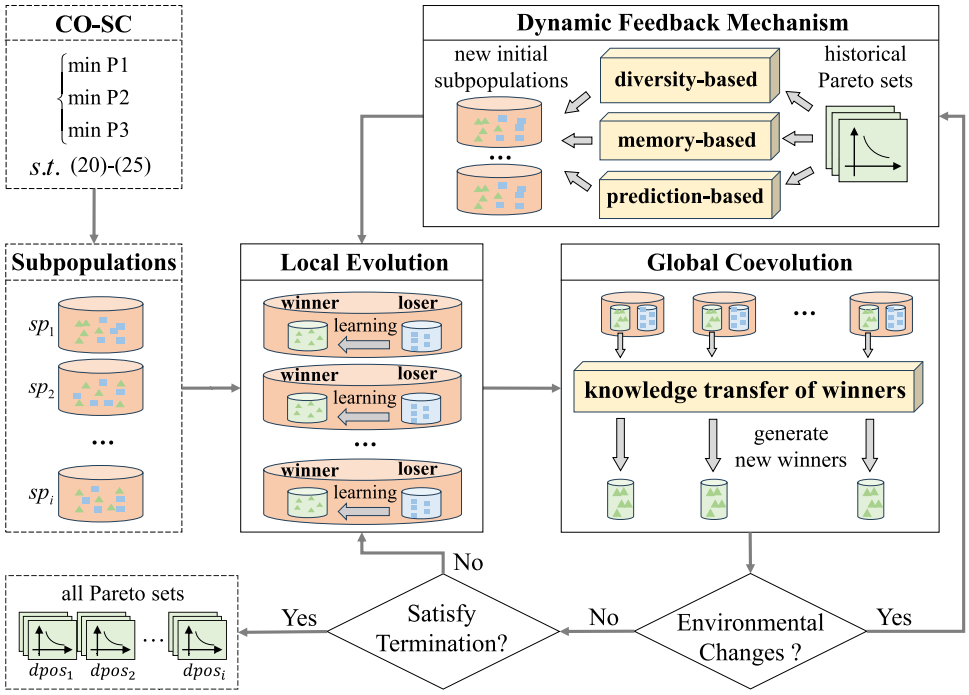


Fig. 2. Outline of the proposed DCMOA for solving the CO-SC problem.

$$x_{ij}, y_{jk}, l_{ik}, a_i^{edge}, a_i^{local}, a_i^{cloud} \in \{0,1\}, \forall i \in I, \forall j \in J. \quad (25)$$

In the formulated model, the primary goal is to minimize the delay, energy consumption, and load balance for each time slot in (19) by optimizing the computation offloading decisions $\{\mathbf{A}, \mathbf{X}\}$ and the service caching decision $\mathbf{Y}$. Constraint (20) guarantees that the computational task of each vehicle can be offloaded to only one ES for edge computing. Constraint (21) ensures that each computational task belongs to only one service type. Constraints (22) and (23) ensure that the storage and computational resources of each ES cannot be exceeded. Constraint (24) ensures that each vehicle can only choose one computing mode. Constraint (25) guarantees the range of values for some related parameters.

However, when solving multi-objective optimization problems, the dimensionality of the search space usually increases with the number of optimization objectives, and it is obviously difficult to find the global optimal solution quickly in a huge search space. To alleviate this issue, we reconstruct F_0 to obtain the following three new subproblems with low-dimensional search space: $F_1 = \min\{P1, P2\}$, $F_2 = \min\{P1, P3\}$, and $F_3 = \min\{P2, P3\}$, each of which optimizes different pairwise objectives to reduce the complexity of the original problem. In addition, to preserve the global information of the original search space, the fourth subproblem $F_4 = \min\{P1, P2, P3\}$ is still employed. In this way, useful information mined in the low-dimensional subspace can be used to assist the optimization of original search space, thus accelerating the convergence. In addition, the global information in the original space can in turn guide the search in the low-dimensional subspace, thus avoiding local optimum.

4. Proposed optimization method

4.1. The overall framework

The outline of the proposed DCMOA is provided in Fig. 2. As shown in Fig. 2, the target CO-SC problem with three objectives ($P1$, $P2$, and $P3$) serves as an input to DCMOA. Then, multiple subpopulations are initialized and each subpopulation is responsible for optimizing the corresponding combination of objectives (i.e., F_1, F_2, F_3, and F_4 mentioned in Section 3.5) in **Local Evolution**. Specifically, each subpopulation is divided into a loser set (disadvantaged solutions) and a winner set (advantaged solutions), where each loser particle will learn promising evolutionary directions from the best-fit winner particle to accelerate convergence. Afterwards, to explore the complete structural information of the search space, the winner particles from different subpopulations will be interacted for knowledge transfer in **Global Coevolution**, which can enhance the diversity and quality of subpopulations by sharing useful knowledge. By combining **Local Evolution** and **Global Coevolution**, DCMOA can efficiently identify the optimal solutions in a stable environment. When environmental changes are detected, **Dynamic Feedback Mechanism** (**DFM**) is designed to generate superior initial subpopulations through three different feedback approaches, which can effectively respond to environmental changes.

Specifically, the pseudocode of the proposed DCMOA is given in **Algorithm 1** with the target CO-SC problem as the input. First, initialize i different subpopulations in Line 1 (i is set to 4 in this paper), which are employed to optimize the corresponding subproblems mentioned in Section 3.5 (i.e., F_1, F_2, F_3, and F_4). Then, in Line 2, two empty sets are initialized: **DPOS**, which preserves the historical DPOS of each subpopulation, **Winner** that includes the winner particles (advantage solutions), and the discrete-time t is set to 0. If the termination condition is not met in Line 3, the evolutionary process is implemented iteratively in Lines 4–18. Particularly, **Local Evolution** and **Global Coevolution** procedures are executed sequentially to identify the optimal solutions for the target CO-SC in Lines 4–14 until environmental change occurs. Specifically in Line 5, each subpopulation sp_i is first separated into two groups (i.e. $winners_i$ and $losers_i$) of equal size through the nondominated relationship (Ali et al., 2022) and crowding distances (Yue et al., 2021), where $winners_i$ denotes the set of solutions with better performance and $losers_i$ indicates the set of those with worse performance. Then, in Line 6, $losers_i$ of sp_i is updated by learning from $winners_i$ through **Local Evolution**, the details of which will be

Algorithm 1 DCMOA

Input: The target CO-SC problem.
Output: The set of DPOS in different environments: **DPOS**.
1: initialize subpopulation **SP** = $\{sp_1, sp_2, ..., sp_i\}$;
2: initialize **DPOS** = $\{dpos_1, dpos_2, ..., dpos_i\}$, $dpos_i = \emptyset$, **Winner** = $\emptyset$, $t = 0$;
3: **while** stopping condition is not met **do**
4: **for all** sp_i **do**
5: $winners_i$, $losers_i \leftarrow$ dividing sp_i;
6: $losers_i$ = **Local Evolution**($winners_i$, $losers_i$);
7: **Winner** = **Winner** $\cup$ $winners_i$;
8: **end for**
9: **Winner** = **Global Coevolution**(**Winner**);
10: **for all** sp_i **do**
11: update each sp_i using $winners_i$ in **Winner**;
12: $dpos_i^t \leftarrow$ get the dynamic Pareto set of sp_i at t;
13: **DPOS** $\leftarrow dpos_i \cup dpos_i^t$;
14: **end for**
15: **if** environment changes **then**
16: **SP** = **DFM**(**SP**, **DPOS**);
17: $t = t + 1$;
18: **end if**
19: **end while**

presented in Section 4.3. In Line 7, $winners_i$ that has not been updated is added to the set **Winner** and awaits subsequent evolution. After collecting $winners_i$ of all sp_i, the **Winner** is updated via implicit knowledge transfer in **Global Coevolution** in Line 9, the details of which will be introduced in Section 4.4. After that, the $winners_i$ of sp_i is updated by the corresponding individuals in **Winner** in Line 11, followed by obtaining the dynamic Pareto sets of each evolved subpopulations and saving into the set **DPOS** in Lines 12–13. Finally, if an environmental change is detected in Line 15, the **DFM** is triggered and generates new initial subpopulations through three dynamic feedback methods in Line 16, the details of which will be provided in Section 4.5. By iteratively running the above process, DCMOA can obtain all DPOS for the four subpopulations in different environments.

4.2. Initialization

Regarding the subpopulation representation, this paper adopts an equal-length integer encoding to represent each solution. Specifically, the first half of the solution represents the service caching decisions of the ESs, while the second half indicates the offloading decisions of the vehicles. For instance, in an environment with two vehicles and two ESs, where each ES can cache two services, consider a solution represented as (132 421). Here, (13) specifies that the first ES caches services 1 and 3, and (24) denotes that the second ES caches services 2 and 4. The last two digits (21) indicate that the first and second vehicles offload their tasks to the second ES and the first ES, respectively.

As for the subpopulation initialization, $N/4$ solutions are randomly generated to form an initial subpopulation. Moreover, four subpopulations (i.e., $SP = \{sp_1, sp_2, sp_3, sp_4\}$) are generated in this paper to optimize the four subproblems (i.e., F_1, F_2, F_3, and F_4) mentioned in Section 3.5.

4.3. Local evolution

The purpose of **Local Evolution** is to obtain appropriate solutions before environmental changes by fully exploring and exploiting the useful knowledge of the subpopulations themselves. Inspired by competitive swarm optimizer (CSO) (Cheng & Jin, 2015), the disadvantaged particles (i.e., *losers*) of the subpopulation observe and learn the behavior of superior particles (i.e., *winners*) with better performance to

Algorithm 2 Local Evolution

Input: $winners_i$, $losers_i$ of the i-subpopulation.
Output: The updated $losers_i$.
1: **for** j : 1 to $|losers_i|$ **do**
2: **for** k : 1 to $|winners_i|$ **do**
3: get $\mathbf{A}(jk)$ between $losers_i(j)$ and $winners_i(k)$;
4: **if** $Min_{\mathbf{A}} > \mathbf{A}(jk)$ **then**
5: $Min_{\mathbf{A}} = \mathbf{A}(jk)$;
6: $ind = k$;
7: **end if**
8: **end for**
9: match $losers_i(j)$ to $winners_i(ind)$ with $Min_{\mathbf{A}}$;
10: update $losers_i(j)$ using (26) and (27);
11: **end for**

improve their qualities. This learning process facilitates knowledge transfer from the *winners* to the *losers*, which promotes the overall improvement of the subpopulation. Specifically, the learning process of *losers* to *winners* is run by the following equation,

$$V_l(h+1) = r_1 \times V_l(h) + r_2 \times \left(X_w(h) - X_l(h)\right) + \varphi \times r_3 \times (\bar{X}(h) - X_l(h)), \tag{26}$$

$$X_l(h+1) = X_l(h) + V_l(h+1), \tag{27}$$

where r_1, r_2, r_3 are three random numbers ranging from 0 to 1. h represents the hth iteration. $V_l(h)$ and $X_l(h)$ denote the velocity and position of *losers*. $X_w(h)$ is the position of *winners*. $\bar{X}(h)$ is the mean position of all solutions in the hth iteration and φ is a control parameter of $\bar{X}(h)$. However, unlike the random pairwise competition in traditional CSO, DCMOA matches the *winners* with the *losers* based on the angular information among particles, in which the two particles with the smallest angle are grouped. In fact, the smaller angle between two particles, the closer they are regarding the position and orientation in the search space. By aligning the evolution direction with the best-fit *winners*, the *losers* can benefit from the knowledge and experiences of those successful particles, leading to faster convergence towards the DPOF. Therefore, using angular information in DCMOA provides a more targeted and efficient learning mechanism compared to the random pairwise competition.

The evolutionary process of **Local Evolution** is presented in **Algorithm 2** with the inputs: the $winners_i$ and $losers_i$ of the i-subpopulation. First, the angles of particles in $winners_i$ and $losers_i$ are calculated in Lines 1–8. Note that $|losers_i|$ and $|winners_i|$ denote the number of loser and winner particles, respectively. Then, each particle in $losers_i$ will match the particle in $winners_i$ that has the smallest angle to it in Line 9. After that, in Line 10, the velocity and position of the loser particle $losers_i(j)$ are updated by learning from its matched winner particle $winners_i(ind)$ through (26) and (27), respectively. Finally, the updated $losers_i$ is returned to replace the corresponding $losers_i$ in the original i-subpopulation.

4.4. Global coevolution

Although **Local Evolution** can reduce the complexity of the optimization task and improve the search efficiency, it may fall into a local optimum or converge prematurely since only part of the search space is explored. Therefore, in the proposed DCMOA, **Global Coevolution** is designed to effectively explore and transfer latent knowledge across subpopulations, which aims to increase the diversity of subpopulations and improve the quality of solutions.

There are some overlapping solution spaces among subpopulations, as the same objective function exists in the optimized target. As a result, the implicit knowledge and experience among subpopulations can be shared with each other to better explore these

Algorithm 3 Global Coevolution

Input: The set of $winners_i$ in all subpopulations: **Winner**.
Output: The updated **Winner**.
1: **for all** $winners_i$ in **Winner do**
2: $\mathbf{W_{ex}} \leftarrow \{\mathbf{x}|\mathbf{x} \in \mathbf{Winner}\ \&\ \mathbf{x} \notin winners_i\}$;
3: $\mathbf{W_{nd}} \leftarrow$ get nondominated solutions from $\mathbf{W_{ex}}$;
4: **for** k : 1 to $|winners_i|$ **do**
5: $\mathbf{x} \leftarrow$ randomly select a solution from the $\mathbf{W_{nd}}$;
6: $winners_i(k) \leftarrow$ crossover $winners_i(k)$ and $\mathbf{x}$;
7: **end for**
8: **end for**

overlapping spaces. Moreover, it is worth noting that there are also distinct solution spaces between subpopulations. By sharing knowledge between diverse sources, a wide range of perspectives can be integrated, thereby facilitating the exploration of untapped regions within the search space. In our designed **Global Coevolution**, genetic information between subpopulations is integrated to explore the overlapping search spaces as well as untapped regions. Specifically, to better utilize the knowledge from external subpopulations to guide the evolution of the i-subpopulation, high-quality individuals (nondominated solutions) from *winners* of external subpopulations are extracted according to the nondominated relationship. After that, *winners* of the i-subpopulation (i.e., $winners_i$) are randomly combined with the external high-quality individuals using crossover operations (Kora & Yadlapalli, 2017) to improve the quality of solutions and increase the diversity of i-subpopulation.

To clarify the running process of **Global Coevolution**, its pseudocode is given in **Algorithm** 3 with the following input: a set **Winner** containing *winners* from all subpopulations (indexed via subscript i, i = 1, 2, 3, 4). First, *winners* of external subpopulations are all collected in $\mathbf{W_{ex}}$ as an external experience pool in Line 2. Then, a set of nondominated solutions $\mathbf{W_{nd}}$ is extracted from $\mathbf{W_{ex}}$ in Line 3. After that, in Lines 4–7, each individual in $winners_i$ exchanges genetic information with a superior individual in $\mathbf{W_{nd}}$ using crossover to achieve transfer knowledge. Finally, after all of $winners_i$ in **Winner** complete knowledge transfer through Lines 2–8, the updated **Winner** is returned.

4.5. Dynamic feedback mechanism

By introducing **Local Evolution** and **Global Coevolution** into DCMOA, the high-quality CO-SC decisions can be obtained in a static VEC system. However, most studies stop there and do not further discuss the impact of dynamic factors. In practice, due to the dynamics of vehicle networks, these high-quality CO-SC decisions may become inferior or infeasible in the new environment, thus requiring massive computational resources and time costs for re-searching. To address the above issue, when environmental changes are detected, a dynamic feedback mechanism **DFM** with three different feedback approaches is introduced in the proposed DCMOA, aiming to rapidly generate high-quality initial subpopulations for the new environments, thus achieving more efficient optimization efficiency than searching from scratch in each new environment.

Specifically, **DFM** incorporates three feedback approaches, including diversity-based, memory-based, and prediction-based approaches. First, the diversity-based approach is adopted to maintain diversity by introducing randomly generated initial solutions. In addition, the memory-based approach is employed to retain the historical solutions with good convergence for the new environment. The above two methods may not perform well in a dramatically changing environment because they ignore historical change patterns. Hence, the prediction-based approach is designed to track the changing DPOS of the subpopulation according to the distribution information of all historical DPOS. In fact, the historical DPOS, which reflects the set

Algorithm 4 Dynamic Feedback Mechanism (DFM)

Input: All subpopulation: $\mathbf{SP} = \{sp_1, sp_2, ..., sp_i\}$; All historical DPOS: $\mathbf{DPOS} = \{dpos_1, dpos_2, ..., dpos_i\}$.
Output: The new initial $\mathbf{SP} = \{sp_1, sp_2, ..., sp_i\}$.
1: **for all** sp_i **do**
2: $\mathbf{sp_{div}} \leftarrow$ randomly generate $|sp_i|/3$ solutions;
3: $\mathbf{sp_{mem}} \leftarrow$ randomly select $|sp_i|/3$ solutions from sp_i;
4: $p^{\uparrow}, p^{\downarrow} \leftarrow$ divide $dpos_i$ by nondominated relationship;
5: $Model \leftarrow$ training Bayesian classifier using $p^{\uparrow}$, $p^{\downarrow}$;
6: $X_{test} \leftarrow$ generate numerous random solutions;
7: $Y_{test} \leftarrow$ predict the class of X_{test} with trained $Model$;
8: $\mathbf{sp_{pre}} \leftarrow \{\mathbf{x}|\mathbf{x} \in X_{test}\ \&\ Y_{test}(\mathbf{x}) = \text{“+1”}\}$;
9: $\mathbf{sp_{pre}} \leftarrow$ select $|sp_i|/3$ solutions from $\mathbf{sp_{pre}}$;
10: $sp_i = \mathbf{sp_{div}} \cup \mathbf{sp_{mem}} \cup \mathbf{sp_{pre}}$;
11: **end for**

of optimal solutions for the CO-SC decisions at different time points, reveals invariant hidden factors and distribution patterns, such as the invariant location of ESs and the positional changes of the vehicle. Analyzing the relationships of DPOS can excavate the impact of these factors and patterns on decision making and provide guidance for obtaining new high-quality CO-SC decisions. In this paper, the naive Bayesian classifier (Leung et al., 2007) is used to analyze historical DPOS for predicting some superior initial solutions. Finally, by combining the above three dynamic feedback methods, multiple superior initial subpopulations are obtained, which will be optimized to find the optimal CO-SC decision for the new environment.

Details of **DFM** are described in **Algorithm** 4 with the following inputs: a set **SP** containing all subpopulations and a set **DPOS** including all historical DPOS of each subpopulation. First, the diversity-based and memory-based approaches are executed in Lines 2 and 3, respectively. The diversity-based approach randomly generates $|sp_i|/3$ initial solutions as $\mathbf{sp_{div}}$, while the memory-based approach retains $|sp_i|/3$ solutions as $\mathbf{sp_{mem}}$ from sp_i, where $|sp_i|$ denotes the size of sp_i. Moreover, the prediction-based approach is implemented in Lines 4–9. Specifically, in Line 4, the set $dpos_i$ of sp_i is separated into two groups: $p^{\uparrow}$ (nondominated solutions) and $p^{\downarrow}$ (dominated solutions). After that, in Line 5, $p^{\uparrow}$ and $p^{\downarrow}$ are used as positive and negative samples to train the Bayesian classifier. Note that the samples labeled with $p^{\uparrow}$ are set to “ +1” and the samples labeled with $p^{\downarrow}$ are set to “−1” during the training of the Bayesian classifier. After the training of the classifier is completed, in Line 6, a large number of solutions are randomly generated and served as a test set X_{test}. In Line 7, the label Y_{test} of X_{test} is predicted by the trained Bayesian classifier. In Lines 8 and 9, the solutions labeled “ +1” are selected from X_{test} as $\mathbf{sp_{pre}}$, where the size of $\mathbf{sp_{pre}}$ is $|sp_i|/3$. In Line 10, the initial solutions obtained by three different approaches are combined into a new initial subpopulation. Finally, all new initial subpopulations are generated according to the above three approaches and returned.

5. Performance evaluation

5.1. Dataset description

To evaluate the performance of the proposed algorithm, two real-world datasets are used in our experiments: Shanghai Telecom’s base station dataset (Li, Zhou et al., 2021; Wang et al., 2019) and Shanghai cab trajectory dataset (Zhao et al., 2022). Specifically, the Shanghai Telecom base station dataset contains the exact location information of 3233 base stations, and the internet access information of mobile users passing through these base stations. The Shanghai cab trajectory dataset contains the trajectories of 4328 cabs on February 20, 2007 in Shanghai. The trajectory of each cab contains exact location information that is recorded every one or half minutes for one day. Fig. 3 displays the distribution of base stations and cab trajectories in the adopted datasets.

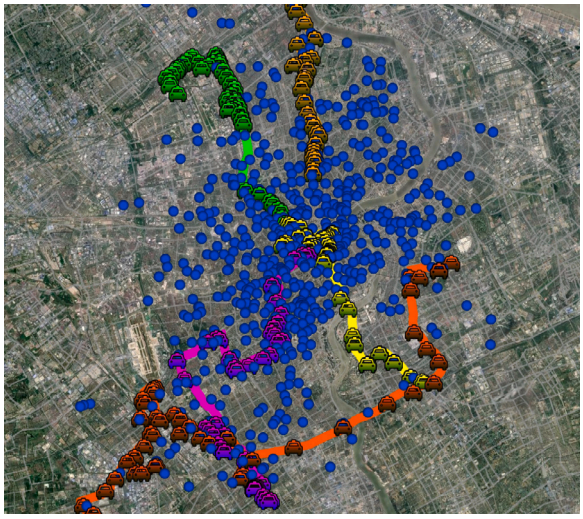


Fig. 3. Distribution of base stations and cab trajectories in Shanghai, where the blue nodes are base stations and five colored lines are the cab trajectories.

Table 3
Environmental settings.

Parameter	Value
Number of ESs	50
Number of vehicles	**20**, 40, 60, 80, 100
Number of service types	8
Number of service types on one ES	3
Time slots/Number of environmental changes	10, 20, 30, 40, **50**
Transmission data of one task	[50, 300] KB
Transmission data of service	5 MB
The CPU cycles to compute one bit	[10, 200] Mcycles
The maximum delay of one task	[200, 400] ms
The bandwidth between vehicle and ES	100 Mbps
The bandwidth between ES and cloud	1 Gbps
The computation capabilities of vehicle	2 GHz
The computation capabilities of ES	20 GHz
The computation capabilities of cloud	200 GHz
The computing energy efficiency parameter	10^{-26}
The average channel transmission power	20 dBm
The average channel gain	15 dBi
The power of the white Gaussian noise	−174 dBm/Hz

5.2. Experimental setup

5.2.1. Environmental setup

From the 3233 real base stations in Shanghai, 50 base stations are selected as the edge servers (ESs) for the VEC system using a K-means clustering algorithm (Guo et al., 2020). Moreover, 100 cabs are randomly selected as moving vehicles from the Shanghai cab trajectory dataset and 50 positions of each cab are sampled from the trajectory as 50 environmental changes. With reference to Li et al. (2024), the main environmental settings for the VEC system are listed in Table 3. According to Alameddine et al. (2019), Moubayed et al. (2020), and Pham et al. (2020), eight real services and their latency requirements are used in our experiments, as shown in Table 4.

5.2.2. Compared algorithms

In the simulation experiments, seven representative algorithms are adopted, including four multi-objective optimization algorithms (i.e., DNSGAII (Deb et al., 2007), LMOCSO (Tian et al., 2020), LMPSO (Gao et al., 2022), and NSGAIII-ASF&WD (Cui et al., 2024)) and three heuristic algorithms (i.e., SCTOA (Liao et al., 2024), NEBS, and RA). Specifically, when the environment changes, similar to DNSGAII, 80% of the solutions of the LMOCSO, LMPSO, and NSGAIII-ASF&WD are retained in the new environment and the remaining are randomly generated. Moreover, SCTOA is a heuristic design, where service caching decisions are done through the least recent used algorithm and each task selects the most economical decision for offloading. NEBS is a greedy algorithm that offloads the computational tasks to the nearest ES for execution, and then the ES selects the appropriate service for caching based on the number of services. RA is a randomized strategy that randomly generates computation offloading and service caching decisions.

5.2.3. Parameter setup

The computational resources required by the algorithm for convergence are influenced by the number of decision variables. Therefore, if the number of vehicles is less than 80, the number of iterations in each time slot is set to 100, otherwise 200. The size of each subpopulation in DCMOA is set to 20. Since there are four subpopulations in DCMOA, the population size of the compared algorithms with a single population is set to 80 (i.e., 20×4). Please note that the parameter settings for the compared algorithms are set according to their original literature.

5.3. Simulation results

This section discusses the experimental results of all the compared algorithms at different environmental scales and numbers, which reflect the dramatic and frequency of environmental changes, respectively. For a fair comparison, each algorithm is independently run 10 times and the average of the experimental results is recorded to evaluate the overall performance of the algorithm.

Fig. 4 shows the experimental results for different scales of vehicles in terms of delay, energy consumption, and load balance, where the number of time slots is set as 50. As illustrated in Fig. 4, the proposed DCMOA performs better than four compared evolutionary algorithms, i.e., DNSGA-II, LMOCSO, LMPSO, and NSGAIII-ASF&WD, all of which employ single population evolution. The reason behind this could be attributed to the multi-subpopulation coevolutionary approach introduced in DCMOA, which can improve the search capability by decomposing the high-dimensional solution space and employing an efficient knowledge transfer strategy. Besides, DCMOA outperforms three heuristic algorithms, i.e., SCTOA, NEBS and RA, which demonstrates that it has an intelligent capability to make an optimal offloading and caching decisions for vehicles and ESs. Finally, it is clear that the three objective values for all compared algorithms gradually increase with the environmental scale, but DCMOA still achieves the best performance in different environmental scale.

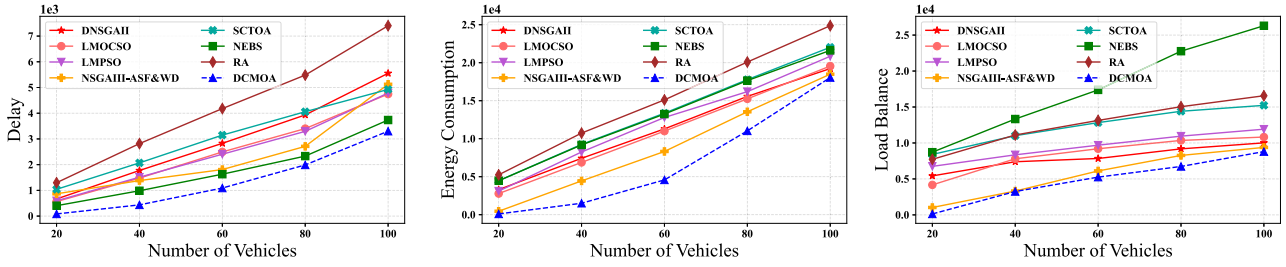
Fig. 5 shows the experimental results for different numbers of time slots in terms of delay, energy consumption, and load balance, where the number of vehicles is set as 20. Overall, DCMOA still outperforms seven compared algorithms for different degrees of environmental changes. As the environment changes, seven competitors of DCMOA get improvement only on delay and evolve slowly on the other two objectives. In contrast, our proposed DCMOA can evolve continuously with environmental changes in all three objectives. This is because DFM smartly maintains the diversity and convergence of solutions via diversity-based and memory-based feedback methods. Meanwhile, by analyzing the distribution information of historical solutions via a prediction-based method, DCMOA is able to respond the environmental changes more effectively, thus improving the optimization performance.

5.4. Ablation analysis

To further validate the effectiveness of the multi-subpopulation coevolutionary approach and DFM in DCMOA, two variants of DCMOA are implemented in our ablation experiments. Specifically, DCMOA-I indicates that DFM is not used and all individuals are retained in the new environment, while DCMOA-II means that the multi-subpopulation coevolutionary approach is not utilized but a single population is used to optimize the three objectives simultaneously. For a fair comparison,

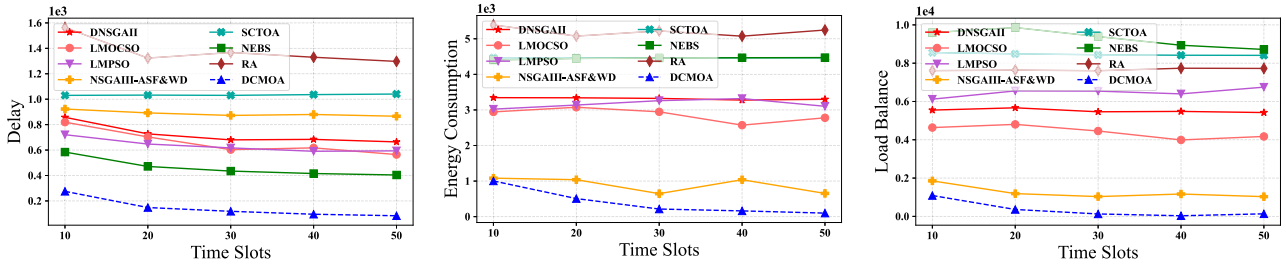
Table 4
Latency requirements of services.

Service type	Latency
Cooperative awareness basic service (Moubayed et al., 2020)	[10, 20] ms
Decentralized environmental notification message (Moubayed et al., 2020)	[10, 100] ms
Media downloading and streaming (Moubayed et al., 2020)	[10, 1000] ms
VR services (Pham et al., 2020)	[10, 100] ms
Cloud gaming (Pham et al., 2020)	[10, 150] ms
Simple web services (Pham et al., 2020)	[10, 1000] ms
Intelligent transportation systems (Alameddine et al., 2019)	[10, 100] ms
Smart grid (Alameddine et al., 2019)	[10, 20] ms



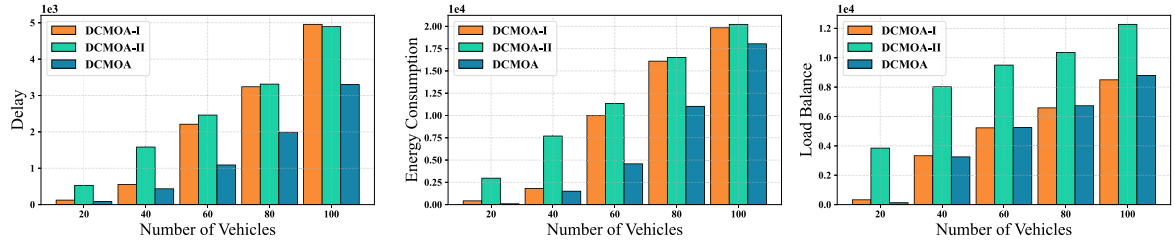
(a) Delay (b) Energy Consumption (c) Load Balance

Fig. 4. Experimental results for different scales of vehicles in terms of (a) delay, (b) energy consumption, and (c) load balance.



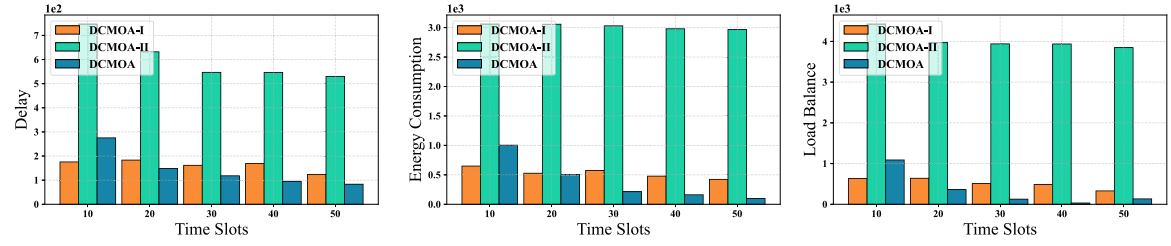
(a) Delay (b) Energy Consumption (c) Load Balance

Fig. 5. Experimental results for different numbers of time slots in terms of (a) delay, (b) energy consumption, and (c) load balance.



(a) Delay (b) Energy Consumption (c) Load Balance

Fig. 6. Ablation experimental results for different scales of vehicles in terms of (a) delay, (b) energy consumption, and (c) load balance.



(a) Delay (b) Energy Consumption (c) Load Balance

Fig. 7. Ablation experimental results for different numbers of time slots in terms of (a) delay, (b) energy consumption, and (c) load balance.

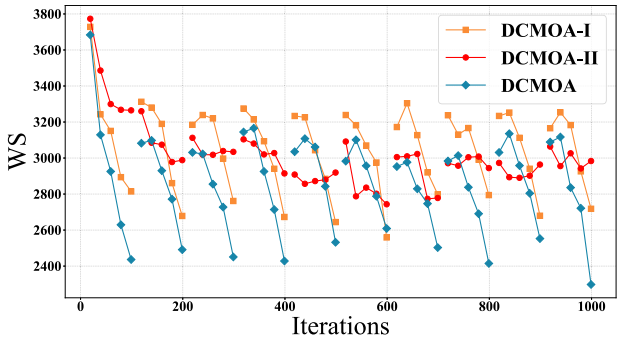


Fig. 8. The WS results for DCMOA and the two variants over ten environmental changes with 100 iterations per change.

the relevant parameters for these two variants are kept the same as the original DCMOA.

Figs. 6 and 7 show the ablation experimental results for different numbers of vehicles and time slots, respectively. In Fig. 6, DCMOA-I performs well in the first two small-scale cases, but deteriorates on the large-scale cases, especially in terms of delay and energy consumption. Meanwhile, in Fig. 7, DCMOA-I evolves slowly with continued environmental changes compared to DCMOA and DCMOA-II. This indicates that DFM can provide algorithms with better quality solutions in large-scale and continuously changing environments, by increasing diversity and learning historical knowledge. On the other hand, DCMOA-II exhibits inferior performance compared to both DCMOA and DCMOA-I as depicted in Figs. 6 and 7. The main reason behind this is that DCMOA-II lacks the ability to quickly discover appropriate solutions prior to environmental changes. In summary, DCMOA enables efficient evolution during frequent and drastic environmental changes by combining the designed multi-subpopulation coevolutionary approach and DFM.

To further validate the impact of two key components on the optimization results, this study employs a key metric known as the weighted sum (WS) of the three optimization objectives at the ideal point of the Pareto frontier, calculated as follows:

$$WS = \alpha_1 f_1(x^*) + \alpha_2 f_2(x^*) + \alpha_3 f_3(x^*), \tag{28}$$

where f_1, f_2, and f_3 denote delay, energy consumption and load balancing respectively. The weights α_1, α_2, and α_3 correspond to these objectives and are all set to 0.3 in this paper. Fig. 8 illustrates the WS results for DCMOA and its two variants over ten environmental changes with 100 iterations per change. At the moment when the environment changes, the WS values for DCMOA-I are consistently worse than those of DCMOA and DCMOA-II, both of which utilize the designed DFM. This observation indicates that DFM is effective in producing high-quality solutions that accelerate algorithm convergence in dynamic environments. Conversely, DCMOA-II tends to fall into a local optimum during static conditions, resulting in poorer final WS values compared to DCMOA and DCMOA-I, which benefit from the multi-subpopulation coevolutionary approach. In summary, the WS results in Fig. 8 further validate the effectiveness of the DFM and the multi-subpopulation coevolutionary approach designed in DCMOA in terms of enhancing convergence speed and avoiding local optima.

6. Limitations

In this paper, we study the CO-SC problem in dynamic VEC systems and propose a dynamic coevolutionary multi-objective optimization algorithm (DCMOA). However, this paper still has some limitations regarding the designed VEC system and method. First, the current VEC system restricts each vehicle to executing only a single computational task each time. In reality, vehicles often tend to handle multiple tasks of various types, which could require more complex service orchestration. Second, the dynamic feedback mechanism (DFM) in DCMOA currently applies three feedback methods in equal proportions. We acknowledge that dynamically adjusting the proportions or using different combinations of feedback methods could better suit various environmental changes, potentially further improving the adaptability of the algorithm. Addressing these limitations in future work could further improve the effectiveness and flexibility of our proposed approach, making it more applicable to real-world dynamic VEC environments.

7. Conclusion and future work

In this paper, we studied the joint computation offloading and service caching (CO-SC) optimization problem in a dynamic VEC system. First, the CO-SC problem is formulated as a dynamic multi-objective optimization problem (DMOP) with the goal of minimizing delay, energy consumption, and load balance. Then, a dynamic coevolutionary multi-objective optimization algorithm (DCMOA) with a multi-subpopulation coevolutionary approach and an efficient dynamic feedback mechanism is proposed to solve the formulated problem. Finally, extensive experiments with real-world datasets in Shanghai, China, are conducted to evaluate the performance of DCMOA, which shows that DCMOA is both effective and well-performed.

For future work, we will further consider computation sharing techniques between vehicles, which can effectively reduce task uploading costs. Moreover, in this work, each vehicle is assumed to generate only one task. We believe that considering the CO-SC problem with multitasking vehicles is an interesting step forward. Finally, we believe that the designed DCMOA can be extended to other dynamic multi-objective optimization problems, such as dynamic resource allocation in cloud computing.

CRediT authorship contribution statement

Qijie Qiu: Methodology, Writing – original draft, Software, Data curation. **Yulong Ye:** Methodology, Writing – original draft. **Lingjie Li:** Conceptualization, Writing – review & editing. **Zhijiao Xiao:** Supervision, Validation, Writing – review & editing. **Qiuzhen Lin:** Supervision, Validation, Project administration. **Zhong Ming:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Natural Science Foundation of Guangdong Province under grant 2023A1515011296; in part by the Stable Support Project of Shenzhen (Project No. 20231120145719001); in part by the Scientific Research Capacity Enhancement Program for Key Construction Disciplines in Guangdong Province under Grant 2024ZDJS063; in part by the Shenzhen Technology University School-level Research Project under Grant 20251061020002; in part by the Key Laboratory of Computing Power Network and Information Security, Ministry of Education under Grant No. 2024PY002; in part by the National Natural Science Foundation of China (NSFC) under Grants 62272315 and 62376163.

Data availability

No data was used for the research described in the article.

References

Alameddine, H. A., Sharafeddine, S., Sebbah, S., Ayoubi, S., & Assi, C. (2019). Dynamic task offloading and scheduling for low-latency IoT services in multi-access edge computing. *IEEE Journal on Selected Areas in Communications*, *37*(3), 668–682.

Ali, I. M., Sallam, K. M., Moustafa, N., Chakraborty, R., Ryan, M. J., & Choo, K.-K. R. (2022). An automated task scheduling model using non-dominated sorting genetic algorithm II for fog-cloud systems. *IEEE Transactions on Cloud Computing*, *10*(4), 2294–2308.

Ameena, B., & Ramasamy, L. (2025). Drawer Cosine optimization enabled task offloading in fog computing. *Expert Systems with Applications*, *259*, Article 125212.

Cheng, R., & Jin, Y. (2015). A competitive swarm optimizer for large scale optimization. *IEEE Transactions on Cybernetics*, *45*(2), 191–204.

Cui, Z., Shi, X., Zhang, Z., Zhang, W., & Chen, J. (2024). Many-objective joint optimization of computation offloading and service caching in mobile edge computing. *Simulation Modelling Practice and Theory*, *133*, Article 102917.

Deb, K., Udaya Bhaskara, R. N., & Karthik, S. (2007). Dynamic multi-objective optimization and decision-making using modified NSGA-II: A case study on hydro-thermal power scheduling. In *International conference on evolutionary multi-criterion optimization* (pp. 803–817). Springer.

Duan, S., Lyu, F., Wu, H., Chen, W., Lu, H., Dong, Z., & Shen, X. (2024). Moto: Mobility-aware online task offloading with adaptive load balancing in small-cell MEC. *IEEE Transactions on Mobile Computing*, *23*(1), 645–659. http://dx.doi.org/10.1109/TMC.2022.3220720.

Dutta, N., Patel, S. K., Faragallah, O. S., Baz, M., & Rashed, A. N. Z. (2022). Caching scheme for information-centric networks with balanced content distribution. *International Journal of Communication Systems*, *35*(7), Article e5104.

Fan, W., Han, J., Su, Y., Liu, X., Wu, F., Tang, B., & Liu, Y. (2022). Joint task offloading and service caching for multi-access edge computing in wifi-cellular heterogeneous networks. *IEEE Transactions on Wireless Communications*, *21*(11), 9653–9667.

Farimani, M. K., Karimian-Aliabadi, S., Entezari-Maleki, R., Egger, B., & Sousa, L. (2024). Deadline-aware task offloading in vehicular networks using deep reinforcement learning. *Expert Systems with Applications*, *249*, Article 123622.

Gao, T., Tang, Q., Li, J., Zhang, Y., Li, Y., & Zhang, J. (2022). A particle swarm optimization with Lévy flight for service caching and task offloading in edge-cloud computing. *IEEE Access*, *10*, 76636–76647.

Gao, K., & Xu, L. (2024). Novel strategies based on a gradient boosting regression tree predictor for dynamic multi-objective optimization. *Expert Systems with Applications*, *237*, Article 121532.

Guo, Y., Wang, S., Zhou, A., Xu, J., Yuan, J., & Hsu, C.-H. (2020). User allocation-aware edge cloud placement in mobile edge computing. *Software: Practice and Experience*, *50*(5), 489–502.

Hong, S. H. (2021). Technofutures in stasis: Smart machines, ubiquitous computing, and the future that keeps coming back. *International Journal of Communication*, *15*, 21.

Hosny, K. M., Awad, A. I., Khashaba, M. M., & Mohamed, E. R. (2023). New improved multi-objective gorilla troops algorithm for dependent tasks offloading problem in multi-access edge computing. *Journal of Grid Computing*, *21*(2), 21.

Hu, Y., Zou, J., Zheng, J., Jiang, S., & Yang, S. (2024). A new framework of change response for dynamic multi-objective optimization. *Expert Systems with Applications*, *248*, Article 123344.

Huang, J., Zhang, M., Wan, J., Chen, Y., & Zhang, N. (2024). Joint data caching and computation offloading in UAV-assisted internet of vehicles via federated deep reinforcement learning. *IEEE Transactions on Vehicular Technology*, 1–13. http://dx.doi.org/10.1109/TVT.2024.3429507.

Jiang, S., Li, J., Sang, G., Wu, H., & Zhou, Y. (2024). Vehicular edge computing meets cache: An access control scheme with fair incentives for privacy-aware content delivery. *IEEE Transactions on Intelligent Transportation Systems*, 1–15. http://dx.doi.org/10.1109/TITS.2024.3357522.

Kora, P., & Yadlapalli, P. (2017). Crossover operators in genetic algorithms: A review. *International Journal of Computer Applications*, *162*(10).

Leung, K. M., et al. (2007). Naive bayesian classifier. *vol. 2007*, In *Polytechnic university department of computer science/Finance and risk engineering* (pp. 123–156).

Li, C., Chai, L., Jiang, K., Zhang, Y., Liu, J., & Wan, S. (2023). DNN partition and offloading strategy with improved particle swarm genetic algorithm in VEC. *IEEE Transactions on Intelligent Vehicles*, 1–11. http://dx.doi.org/10.1109/TIV.2023.3346506.

Li, L., Qiu, Q., Xiao, Z., Lin, Q., Gu, J., & Ming, Z. (2024). A two-stage hybrid multi-objective optimization evolutionary algorithm for computing offloading in sustainable edge computing. *IEEE Transactions on Consumer Electronics*.

Li, C., Zhang, Y., Sun, Q., & Luo, Y. (2021). Collaborative caching strategy based on optimization of latency and energy consumption in MEC. *Knowledge-Based Systems*, *233*, Article 107523.

Li, Y., Zhou, A., Ma, X., & Wang, S. (2021). Profit-aware edge server placement. *IEEE Internet of Things Journal*, *9*(1), 55–67.

Li, Y., Zhu, X., Li, N., Wang, L., Chen, Y., Yang, F., & Zhai, L. (2022). Collaborative content caching and task offloading in multi-access edge computing. *IEEE Transactions on Vehicular Technology*, *72*(4), 5367–5372.

Liao, Z., Yin, G., Tang, X., & Liu, P. (2024). A cooperative community-based framework for service caching and task offloading in multi-access edge computing. *IEEE Transactions on Network and Service Management*, *21*(3), 3224–3235. http://dx.doi.org/10.1109/TNSM.2024.3372295.

Liu, W., Xu, X., Qi, L., Zhou, X., Yan, H., Xia, X., & Dou, W. (2024). Digital twin-assisted edge service caching for consumer electronics manufacturing. *IEEE Transactions on Consumer Electronics*, *70*(1), 3141–3151. http://dx.doi.org/10.1109/TCE.2024.3357136.

Masood, A., Dao, N.-N., & Cho, S. (2024). Privacy-preserving intelligent content-caching scheme in heterogeneous aerial access networks. *Expert Systems with Applications*, *243*, Article 122834.

Mirza, M. A., Junsheng, Y., Raza, S., Ahmed, M., Asif, M., Irshad, A., & Kumar, N. (2023). MCLA task offloading framework for 5G-NR-V2X-based heterogeneous VECNs. *IEEE Transactions on Intelligent Transportation Systems*, *24*(12), 14329–14346.

Moubayed, A., Shami, A., Heidari, P., Larabi, A., & Brunner, R. (2020). Edge-enabled V2X service placement for intelligent transportation systems. *IEEE Transactions on Mobile Computing*, *20*(4), 1380–1392.

Pham, X.-Q., Nguyen, T.-D., Nguyen, V., & Huh, E.-N. (2020). Joint service caching and task offloading in multi-access edge computing: A qoe-based utility optimization approach. *IEEE Communications Letters*, *25*(3), 965–969.

Shen, R., Gao, M., Li, W., & Li, Y. (2023). Dynamic task offloading in distributed VEC networks: An exploration and exploitation assisted contract-theoretic approach. *IEEE Transactions on Vehicular Technology*, 1–12. http://dx.doi.org/10.1109/TVT.2023.3332956.

Song, S., Ma, S., Zhu, X., Li, Y., Yang, F., & Zhai, L. (2023). Joint bandwidth allocation and task offloading in multi-access edge computing. *Expert Systems with Applications*, *217*, Article 119563.

Tang, C., Zhu, C., Wu, H., Li, Q., & Rodrigues, J. J. (2021). Toward response time minimization considering energy consumption in caching-assisted vehicular edge computing. *IEEE Internet of Things Journal*, *9*(7), 5051–5064.

Tian, H., Xu, X., Qi, L., Zhang, X., Dou, W., Yu, S., & Ni, Q. (2021). CoPace: Edge computation offloading and caching for self-driving with deep reinforcement learning. *IEEE Transactions on Vehicular Technology*, *70*(12), 13281–13293.

Tian, Y., Zheng, X., Zhang, X., & Jin, Y. (2020). Efficient large-scale multiobjective optimization based on a competitive swarm optimizer. *IEEE Transactions on Cybernetics*, *50*(8), 3696–3708. http://dx.doi.org/10.1109/TCYB.2019.2906383.

Wang, S., Guo, Y., Zhang, N., Yang, P., Zhou, A., & Shen, X. (2019). Delay-aware microservice coordination in mobile edge computing: A reinforcement learning approach. *IEEE Transactions on Mobile Computing*, *20*(3), 939–951.

Wang, C., Sun, H., Wei, L., & Hu, Z. (2024). A novel knee-guided algorithm based on frequency analysis for non-cyclic dynamic multiobjective optimization problems. *Expert Systems with Applications*, *237*, Article 121538.

Wang, P., Wang, Y., Qiao, J., & Hu, Z. (2023). Traffic-aware optimization of task offloading and content caching in the internet of vehicles. *Applied Sciences*, *13*(24), 13069.

Wang, X., Ye, J., & Lui, J. C. (2022). Decentralized task offloading in edge computing: A multi-user multi-armed bandit approach. In *IEEE INFOCOM 2022-IEEE conference on computer communications* (pp. 1199–1208). IEEE.

Xiao, Z., Shu, J., Jiang, H., Lui, J. C. S., Min, G., Liu, J., & Dustdar, S. (2023). Multi-objective parallel task offloading and content caching in D2D-aided MEC networks. *IEEE Transactions on Mobile Computing*, *22*(11), 6599–6615.

Xu, Y., Peng, Z., Song, N., Qiu, Y., Zhang, C., & Zhang, Y. (2024). Joint optimization of service caching and task offloading for customer application in MEC: A hybrid SAC scheme. *IEEE Transactions on Consumer Electronics*, http://dx.doi.org/10.1109/TCE.2024.3443168, 1–1.

Yang, C., Chen, Q., Zhu, Z., Huang, Z.-A., Lan, S., & Zhu, L. (2023). Evolutionary multitasking for costly task offloading in mobile edge computing networks. *IEEE Transactions on Evolutionary Computation*, http://dx.doi.org/10.1109/TEVC.2023.3255266, 1–1.

Yang, Z., Liu, Y., Chen, Y., & Al-Dhahir, N. (2020). Cache-aided NOMA mobile edge computing: A reinforcement learning approach. *IEEE Transactions on Wireless Communications*, *19*(10), 6899–6915.

Yuan, H., Bi, J., Wang, Z., Yang, J., & Zhang, J. (2024). Partial and cost-minimized computation offloading in hybrid edge and cloud systems. *Expert Systems with Applications*, *250*, Article 123896.

Yue, C., Suganthan, P. N., Liang, J., Qu, B., Yu, K., Zhu, Y., & Yan, L. (2021). Differential evolution using improved crowding distance for multimodal multiobjective optimization. *Swarm and Evolutionary Computation*, *62*, Article 100849.

Zhang, J., Chen, S., Wang, X., & Zhu, Y. (2021). DeepReserve: Dynamic edge server reservation for connected vehicles with deep reinforcement learning. In *IEEE INFOCOM 2021-IEEE conference on computer communications* (pp. 1–10). IEEE.

Zhao, X., Su, J., Cai, J., Yang, H., & Xi, T. (2022). Vehicle anomalous trajectory detection algorithm based on road network partition. *Applied Intelligence: The International Journal of Artificial Intelligence, Neural Networks, and Complex Problem-Solving Technologies*, *52*(8), 8820–8838.

Zhao, J., Zhao, W., Deng, B., Wang, Z., Zhang, F., Zheng, W., Cao, W., Nan, J., Lian, Y., & Burke, A. F. (2024). Autonomous driving system: A comprehensive survey. *Expert Systems with Applications*, *242*, Article 122836. http://dx.doi.org/10.1016/j.eswa.2023.122836.

Zheng, G., Xu, C., Wen, M., & Zhao, X. (2022). Service caching based aerial cooperative computing and resource allocation in multi-UAV enabled MEC systems. *IEEE Transactions on Vehicular Technology*, *71*(10), 10934–10947.

Zhou, R., Wu, X., Tan, H., & Zhang, R. (2022). Two time-scale joint service caching and task offloading for uav-assisted mobile edge computing. In *IEEE INFOCOM 2022-IEEE conference on computer communications* (pp. 1189–1198). IEEE.

Zhou, J., & Zhang, X. (2021). Fairness-aware task offloading and resource allocation in cooperative mobile-edge computing. *IEEE Internet of Things Journal*, *9*(5), 3812–3824.

Zhou, H., Zhang, Z., Li, D., & Su, Z. (2023). Joint optimization of computing offloading and service caching in edge computing-based smart grid. *IEEE Transactions on Cloud Computing*, *11*(2), 1122–1132.